1. Project Title: AI-Based Soil Health Assessment System for Nutrient Deficiency Detection and Crop Recommendation

2. Project Statement and Outcomes:

Farmers are frequently unaware of soil nutrient deficiencies and degradation trends, resulting in improper fertilizer usage, poor crop performance, and long-term soil damage. Existing soil diagnostic tools compound this problem by requiring desktop access and English proficiency, making them unusable for the majority of smallholder farmers in India. This project builds a full-stack AI-powered platform that accepts soil image inputs and structured soil test parameters to diagnose soil health conditions and generate targeted remediation recommendations. Using convolutional neural network (CNN) models trained on labeled soil imagery combined with a rule-based and ML recommendation engine, the system identifies deficiencies in nitrogen, phosphorus, potassium, and pH balance, then advises on appropriate fertilizer types, crop suitability, and soil improvement practices. The primary delivery channel is a mobile-first Progressive Web App (PWA) built with Next.js, enabling farmers to photograph soil directly from their Android phone camera, receive AI-generated diagnoses, and read recommendations in Hindi or Kannada — without installing any app from a store.

The platform delivers a farmer-facing PWA where farmers can capture soil photographs directly using their phone camera or enter lab test readings on simple icon-driven forms, and immediately receive AI-generated soil health reports with actionable recommendations in their regional language. Long-term soil degradation trend analysis will help track field health over multiple seasons and plan sustainable soil management programs.

3. Modules to be Implemented:

1. Image & Soil Test Data Ingestion Pipeline
2. ML Soil Classification & Analysis Engine
3. Recommendation Engine, Mobile-First PWA & Reporting
4. System Integration, Testing & Project Finalization

4. Week-wise Module Implementation and High-Level Requirements:

Weeks 1–2: Image & Soil Test Data Ingestion Pipeline

- Build an image upload pipeline supporting JPEG, PNG, and direct camera capture inputs. For the PWA, implement camera capture using the browser MediaDevices API (getUserMedia / input[capture=environment]) so farmers can photograph soil directly from their Android phone without a file picker; include server-side validation, resizing, and format normalisation.
- Design structured input forms for laboratory soil test parameters (NPK levels, pH, organic matter, moisture content).
- Implement OpenCV-based image preprocessing: color correction, background removal, and feature-preserving augmentation for the training pipeline.
- Collect and curate a labeled soil imagery dataset from agricultural institutions and open-source repositories (USDA, ICAR).
- Set up a PostgreSQL database to store soil records, uploaded images, and historical analysis results per farm field.

Weeks 3–4: ML Soil Classification & Analysis Engine

- Train a CNN (ResNet-50 / EfficientNet) on labeled soil images to classify soil type, texture, and visible deficiency symptoms.
- Develop a gradient-boosted classification model using structured soil test data to predict nutrient deficiency categories.
- Implement a hybrid fusion model combining image-based and structured data predictions for improved diagnostic accuracy.
- Build model explainability features using Grad-CAM for visual heatmaps and SHAP for feature importance in structured inputs.
- Serve the model via a FastAPI endpoint returning soil health scores, deficiency diagnoses, and degradation risk indices.

Weeks 5–6: Recommendation Engine, Mobile-First PWA & Reporting

- Build a knowledge-based recommendation engine mapping diagnosed deficiencies to specific fertilizer formulations, dosages, and application schedules.
- Develop a crop suitability ranking module that matches soil health profiles to a curated crop-soil compatibility database.

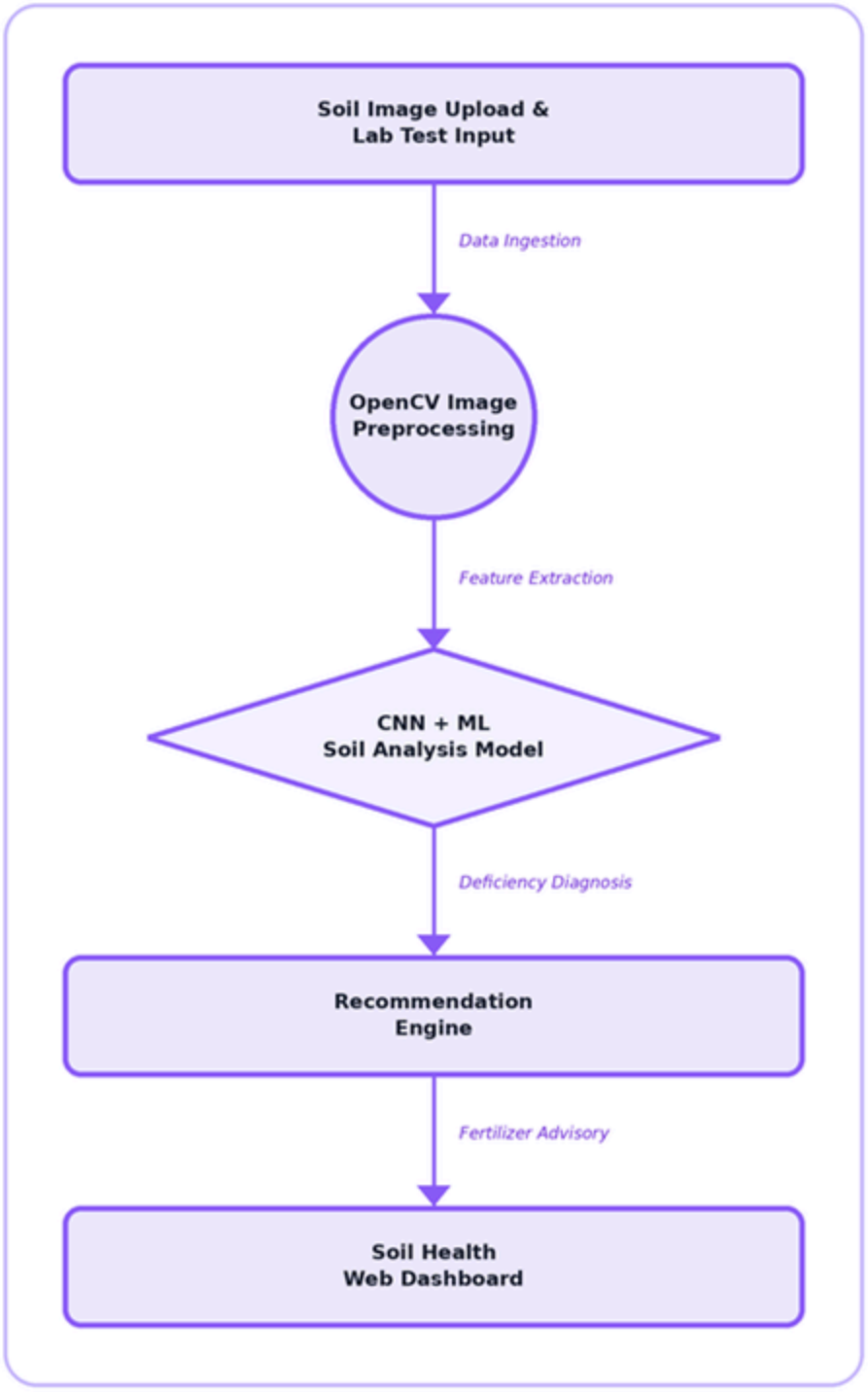
- Implement a long-term soil degradation trend predictor using historical field records and ML regression models.
- Build the farmer-facing PWA using Next.js: scaffold the project with next-pwa (service worker + web app manifest) so it is installable on Android Chrome without an app store. Design icon-driven input screens for soil photo capture (using the browser MediaDevices camera API — note: requires HTTPS and Android Chrome 60+, tested on low-end devices), structured NPK/pH form entry, and a results screen displaying the soil health score, deficiency diagnosis, and crop recommendations as colour-coded cards. Implement next-i18next for static UI strings in Hindi and Kannada, and integrate Sarvam AI for reading AI-generated diagnostic results and recommendations aloud in the farmer's regional language.
- Build a downloadable PDF soil health report generator. Deploy the Next.js PWA and FastAPI backend on cloud infrastructure with secure image storage (AWS S3 / GCS) and CI/CD pipelines; verify the PWA Lighthouse score meets installability requirements (PWA ≥ 90, Performance ≥ 75 on simulated 4G) and that camera capture functions correctly via HTTPS on Android Chrome.

Weeks 7–8: System Integration, Testing & Project Finalization

- Conduct end-to-end integration testing across all modules — image upload → CNN analysis → recommendation engine → soil health dashboard — and resolve any cross-module compatibility issues and edge-case failures.
- Perform load and stress testing on the backend APIs and cross-browser UI validation; address performance bottlenecks and implement enhancements from peer and mentor review.
- Refine the user experience based on testing feedback: improve interface clarity, add contextual help tooltips, and ensure full responsiveness on mobile and tablet devices. Validate PWA camera capture on low-end Android devices and verify Sarvam AI-generated diagnostic readouts in Hindi and Kannada for accuracy and clarity under rural network conditions.
- Prepare comprehensive project documentation covering system architecture, data flow diagrams, API reference, model or algorithm evaluation report, deployment guide, and known limitations.

- Develop the final project presentation (PPT) covering problem statement, proposed solution, architecture, key features, technology stack, results, challenges, and future scope — and rehearse the full end-to-end demo for a smooth, confident delivery.

5. Flow Chart:



6. Sample Dashboard: